

INTERNATIONAL CYBERSECURITY AND DIGITAL FORENSICS ACADEMY

**LAB 1 - NETWORK AND SERVICE RECONNAISSANCE WITH NMAP AND
WHATWEB**

BY

MUSTAPHA ABUBAKAR

ID: C11/26/FCDF/17172

**Course WADF-2026-M04 – LAB 1 - NETWORK AND SERVICE RECONNAISSANCE
WITH NMAP AND WHATWEB**

INSTRUCTOR

MR AMINU IDRIS

Date of Submission 06 SEPTEMBER, 2026

Introduction

This report documents the completion of Lab 1 – Network and Service Reconnaissance with Nmap and WhatWeb, carried out on an authorised, isolated lab environment consisting of a Kali Linux virtual machine (192.168.80.130) and an instructor-authorised Metasploitable 2 target (192.168.80.128). Over the course of the practical, host discovery, service/version detection, OS fingerprinting, full TCP and UDP port scans, NSE script enumeration, and WhatWeb web-technology fingerprinting were performed, revealing 30 open TCP ports and 7 UDP ports on the target. The findings confirm a deliberately outdated, multi-service Linux system consistent with its purpose as a vulnerable training environment, including legacy protocols, anonymous FTP access, and a web stack running Apache 2.2.8 with PHP 5.2.4 — findings that set up the web application testing carried out in Lab 2.

AIM

The aim of this lab is to perform authorised network and service reconnaissance against the Metasploitable 2 target using Nmap and WhatWeb, in order to identify live hosts, open ports, running services, service versions, the likely operating system, and the exposed web technology stack, and to document these findings as a professional reconnaissance record.

OBJECTIVES

- Identify Kali and target IP addresses.
- Understand Nmap command structure and common scan states.
- Perform host discovery, default, version, OS, aggressive, full TCP and basic UDP scans.
- Use safe Nmap NSE scripts to enumerate services.
- Interpret ports, services, versions, banners and HTTP headers.
- Use WhatWeb at multiple aggression levels to fingerprint an authorised web service.

- Create a clear network/service inventory and evidence pack.

TOOLS AND RESOURCES USED

- 🚦 Kali GNU/Linux Virtual Machine (attacker)
- 🚦 Metasploitable 2 Virtual Machine (authorised lab target)
- 🚦 VMware Workstation Player
- 🚦 Nmap 7.94SVN
- 🚦 WhatWeb
- 🚦 curl

METHODOLOGY

The methodology below follows the exact sequence defined in the instructor's laboratory brief.

Each step records the command executed, its purpose, the expected result, and the supporting screenshot evidence.

PART 1 - PREPARE THE LAB

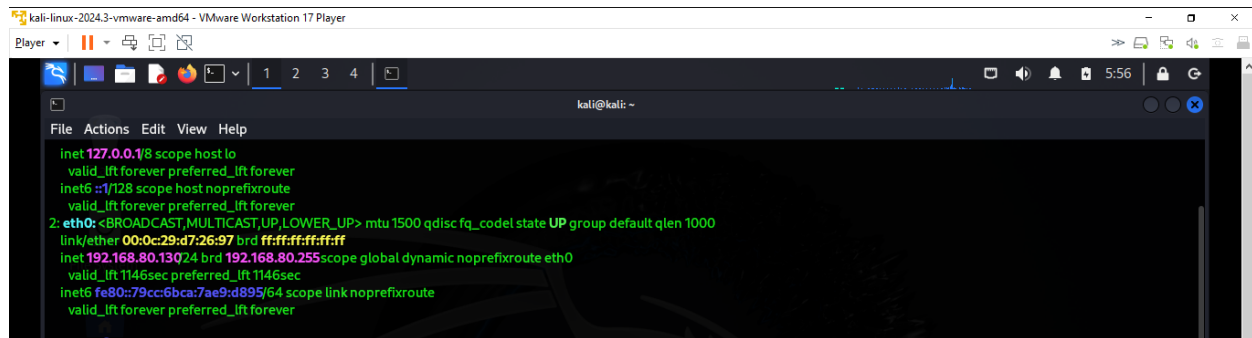
This part identifies the attacker and target addresses and confirms connectivity between the two virtual machines before any scanning begins.

Step 1: Find Kali IP

Objective: Know your own lab address.

Command Used:

ip addr



```

kali@kali: ~
File Actions Edit View Help

inet 127.0.0.1 scope host lo
    valid_lft forever preferred_lft forever
inet6 ::1 scope host noprefixroute
    valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 00:0c:29:d7:26:97 brd ff:ff:ff:ff:ff:ff
    inet 192.168.80.130/24 brd 192.168.80.255 scope global dynamic noprefixroute eth0
        valid_lft 1146sec preferred_lft 1146sec
    inet6 fe80::79cc:6bca:7ae9:d895/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

```

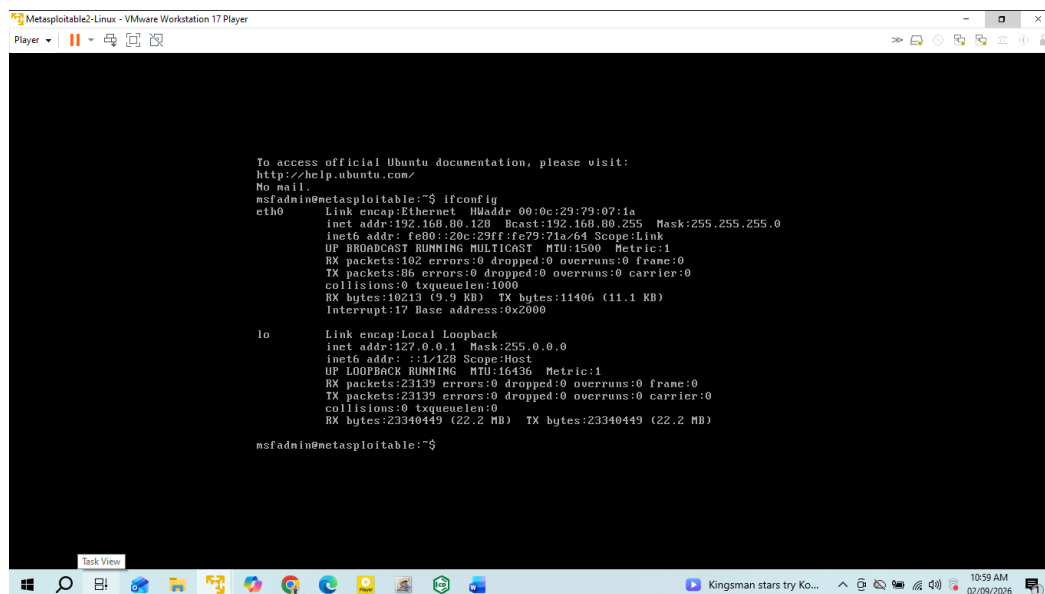
Figure 1. illustrate an ip addr

The command was executed successfully as shown in Figure 1 above.

Step 2: Find Metasploitable 2 IP

Command used

ifconfig



```

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr 00:0c:29:79:07:1a
          inet addr:192.168.0.128  Bcast:192.168.0.255  Mask:255.255.255.0
          inet6 addr: fe80::20c:29ff:fe79:71a/64 Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:102 errors:0 dropped:0 overruns:0 frame:0
          TX packets:86 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:10213 (9.9 KB)  TX bytes:11406 (11.1 KB)
          Interrupt:17 Base address:0x2000

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:23139 errors:0 dropped:0 overruns:0 frame:0
          TX packets:23139 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:23340449 (22.2 MB)  TX bytes:23340449 (22.2 MB)

msfadmin@metasploitable:~$

```

Figure 2. The command was executed successfully as shown in Figure 2 above. Run inside the Metasploitable 2 VM and locate the active interface address.

Step 3: Confirm connectivity

Command used

ping -c 4 <target-ip>

```

kali@kali: ~
File Actions Edit View Help
inet 127.0.0.1 scope host lo
valid_lft forever preferred_lft forever
inet6 ::1/128 scope host noprefixroute
valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
link/ether 00:0c:29:d7:26:97 brd ff:ff:ff:ff:ff:ff
inet 192.168.80.130/24 brd 192.168.80.255 scope global dynamic noprefixroute eth0
valid_lft 1146sec preferred_lft 1146sec
inet6 fe80::79cc:6bca:7ae9:d895/64 scope link noprefixroute
valid_lft forever preferred_lft forever

[kali@kali] ~
$ ping -c 4 192.168.80.128
PING 192.168.80.128 (192.168.80.128) 56(84) bytes of data:
64 bytes from 192.168.80.128: icmp_seq=1 ttl=64 time=0.607 ms
64 bytes from 192.168.80.128: icmp_seq=2 ttl=64 time=0.866 ms
64 bytes from 192.168.80.128: icmp_seq=3 ttl=64 time=0.921 ms
64 bytes from 192.168.80.128: icmp_seq=4 ttl=64 time=1.35 ms

--- 192.168.80.128 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3005ms
rtt min/avg/max/mdev = 0.607/0.936/1.353/0.267 ms

```

Figure 3. The command was executed successfully as shown in Figure 3 above. -c 4 sends four ICMP echo requests then stops automatically. Four replies with time= values and 0% packet loss.

PART 2 - UNDERSTAND NMAP OUTPUT

Before running scans, the meaning of the core Nmap port states and fields was reviewed, summarised in the table below.

Field / State	Meaning
PORT	Port number and protocol, e.g. 22/tcp.
open	A service is accepting connections/traffic.
closed	The host is reachable but nothing is listening on that port.
filtered	Nmap cannot determine the state because filtering blocks probes/responses.
SERVICE	Nmap's service identification.
VERSION	Product/version information when -sV succeeds.

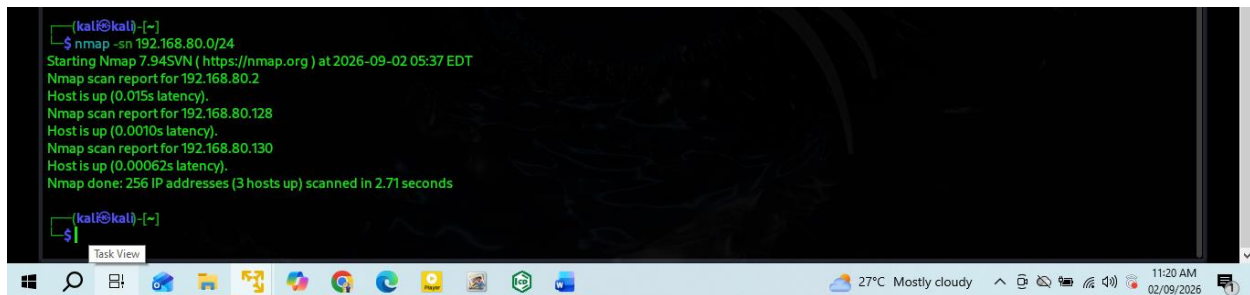
PART 3 - CORE NMAP SCANS

This part performs the full sequence of Nmap scan types required by the lab, moving from a basic host discovery scan through to a complete, versioned TCP and UDP inventory.

Step 4: Host discovery

Command Used

```
nmap -sn 192.168.x.0/24
```

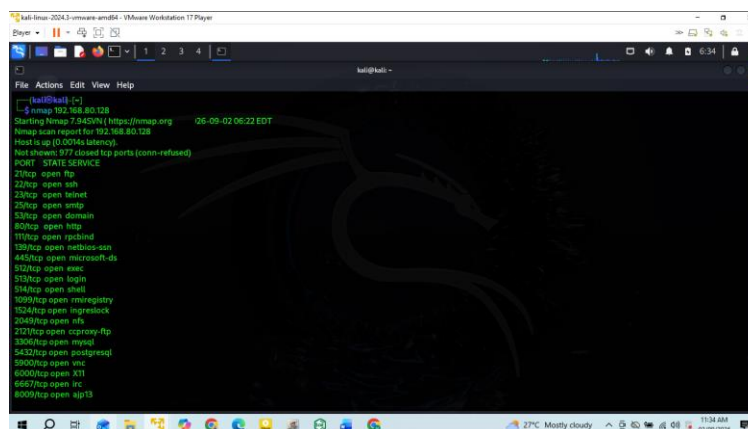


The command was executed successfully as shown in Figure 4 above. -sn performs host discovery only, without scanning ports. Nmap scan report entries for live lab systems.

Step 5: Default TCP scan

Command used

```
nmap <target-ip>
```



The command was executed successfully as shown in Figure 5 above. Nmap scans its default set of the 1000 most common TCP ports. A PORT, STATE and SERVICE table.

Step 6: Service/version detection

Command Used

`nmap -sV <target-ip>`

```

kali@kali: ~
$ nmap -sV 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org 26-09-02 06:35 EDT
Nmap scan report for 192.168.80.128
Host is up (0.0019s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login?
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)
6667/tcp  open  irc          UnrealIRCd
8009/tcp  open  ajp13        Apache Jserv (Protocol v1.3)
  
```

The command `nmap -sV 192.168.80.128` was run successfully, as shown in Figure 6. Unlike the earlier default scan, this scan added a **VERSION** column that shows the exact software and version running behind each open port. Several services were clearly identified, including vsftpd 2.3.4 on port 21, OpenSSH 4.7p1 (Debian 8ubuntu1) on port 22, Postfix smtpd on port 25, Apache httpd 2.2.8 (Ubuntu) on port 80, MySQL 5.0.51a-3ubuntu5 on port 3306, and Apache Tomcat/Coyote 1.1 on port 8180. Two ports — 513 and 514 — were only partially identified at this stage, showing login? and tcpwrapped rather than a confirmed version. This shows that `-sV` is far more useful for real assessment work than a default scan, since knowing the exact software version is what allows a tester to check for known weaknesses later.

Step 7: Maximum version intensity

Command used

`nmap -sV --version-intensity 9 <target-ip>`

```

kali@kali: ~
File Actions Edit View Help
kali@kali: ~
$ nmap -sV --version-intensity 9 192.168.80.128
sudo nmap -O 192.168.80.128
sudo nmap -A 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org 26-09-02 06:36 EDT
Nmap scan report for 192.168.80.128
Host is up (0.0016s latency).
Not shown: 977 closed tcp ports (conn-refused)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn  Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  XT1          (access denied)

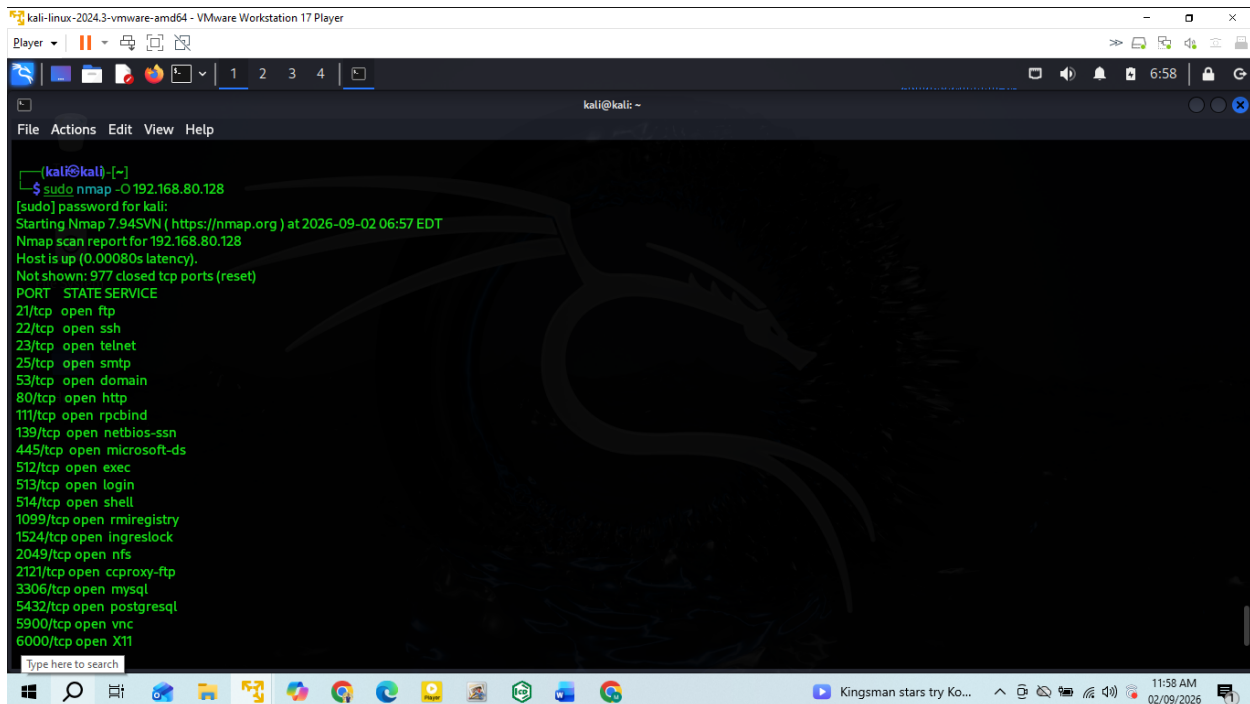
```

The command `nmap -sV --version-intensity 9 192.168.80.128` was run successfully, as shown in Figure 7. This setting tells Nmap to use its most thorough set of version-detection probes instead of the default balance between speed and accuracy. Comparing this result with Step 6, most services returned identical version information, showing the earlier scan was already accurate. The one clear improvement was on port 513, which changed from the unconfirmed login? in Step 6 to a confirmed login OpenSSH or Solaris rlogind service at intensity 9. Port 514 remained tcpwrapped in both scans, meaning Nmap still could not positively identify it even at maximum intensity. Overall, this comparison shows that higher intensity can resolve a few uncertain results, but it comes at the cost of extra scan time, and on a small, well-known target like this one, the improvement was modest.

Step 8: OS detection

Command used

`sudo nmap -O <target-ip>`



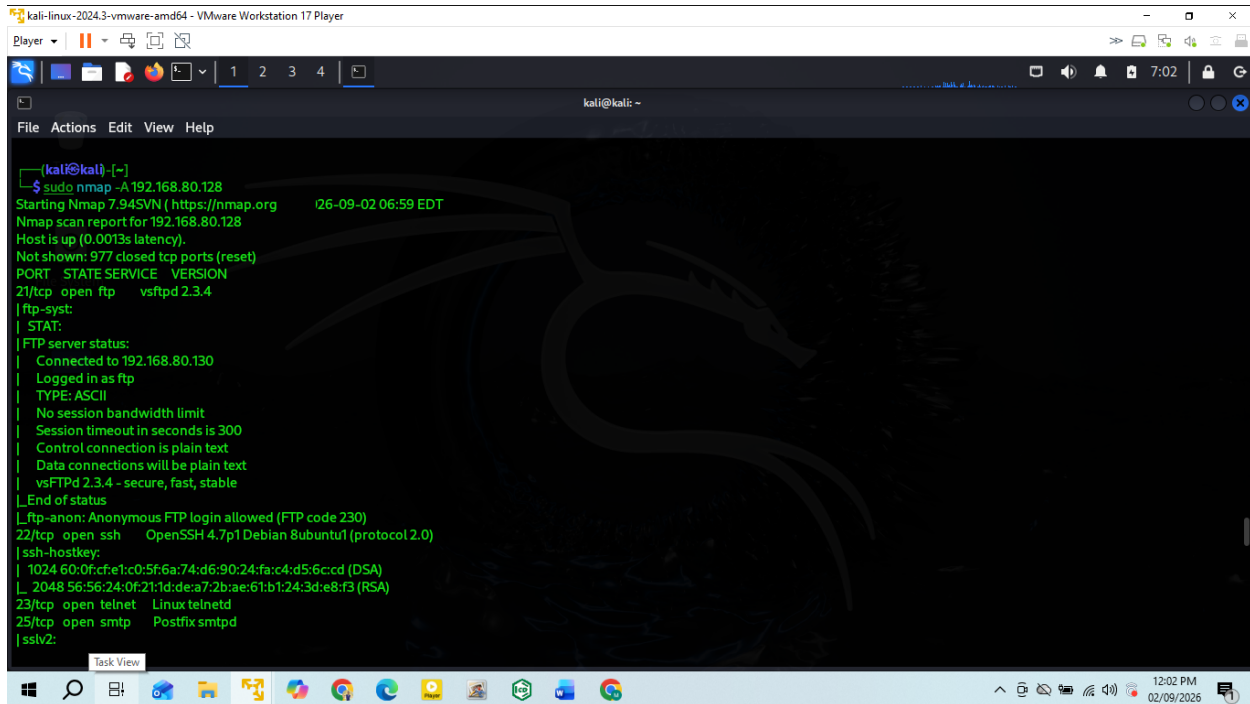
```
(kali@kali) ~  
$ sudo nmap -O 192.168.80.128  
[sudo] password for kali:  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 06:57 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.00080s latency).  
Not shown: 977 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  ccproxy-ftp  
3306/tcp  open  mysql  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11
```

The command was run successfully, as shown in Figure 8. Nmap fingerprinted the target as **Linux 2.6.9 – 2.6.33**, a general-purpose device one hop away, and picked up the VMware MAC address confirming it's a VM. Nmap gave a version range rather than one exact OS, showing OS detection is often an estimate rather than a certainty.

Step 9: Aggressive scan

Command used

`sudo nmap -A <target-ip>`



```

kali@kali: ~
File Actions Edit View Help

(kali@kali) ~
$ sudo nmap -A 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) 26-09-02 06:59 EDT
Nmap scan report for 192.168.80.128
Host is up (0.0013s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE VERSION
21/tcp    open  ftp      vsftpd 2.3.4
| ftp-syst:
|_ STAT:
|_ FTP server status:
|_   Connected to 192.168.80.130
|_   Logged in as ftp
|_   TYPE: ASCII
|_   No session bandwidth limit
|_   Session timeout in seconds is 300
|_   Control connection is plain text
|_   Data connections will be plain text
|_   vsFTPD 2.3.4 - secure, fast, stable
|_ End of status
|_ ftp-anon: Anonymous FTP login allowed (FTP code 230)
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
| ssh-hostkey:
|_ 1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)
|_ 2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)
23/tcp    open  telnet   Linux telnetd
25/tcp    open  smtp      Postfix smtpd
| sslv2:

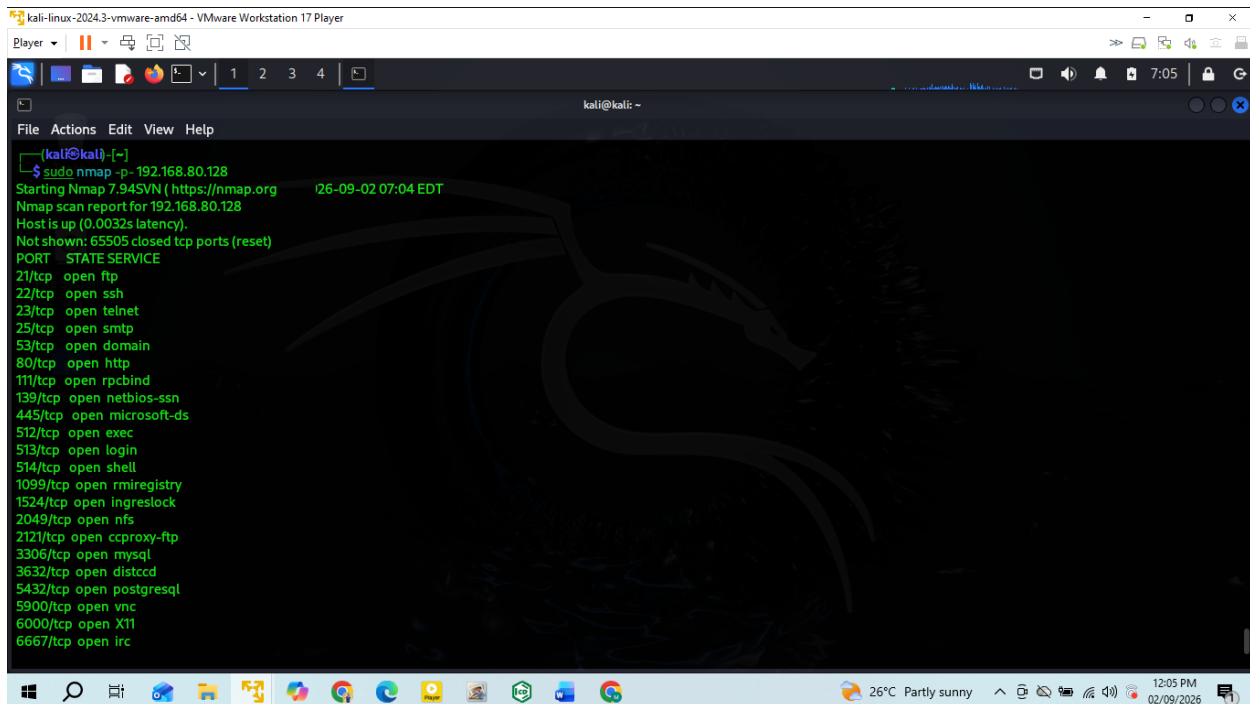
```

The command was run successfully, as shown in Figure 9. Compared to earlier steps, three new findings appeared: anonymous FTP login is allowed on port 21, SMB message signing is disabled (flagged as dangerous), and UnrealIRCd 3.2.8.1 was version-fingerprinted on port 6667. This shows -A combines OS detection, version detection, and script scanning into one deeper pass.

Step 10: Full TCP port scan

Command used

```
sudo nmap -p- <target-ip>
```



The screenshot shows a terminal window titled 'kali@kali: ~' with a dark background and a dragon logo. The command `sudo nmap -p- 192.168.80.128` has been executed. The output displays the Nmap version (7.94SVN), the target IP, and a list of 65,535 open TCP ports. The ports are listed in a table with columns for PORT, STATE, and SERVICE. The ports are: 21/tcp (open ftp), 22/tcp (open ssh), 23/tcp (open telnet), 25/tcp (open smtp), 53/tcp (open domain), 80/tcp (open http), 111/tcp (open rpcbind), 139/tcp (open netbios-ssn), 445/tcp (open microsoft-ds), 512/tcp (open exec), 513/tcp (open login), 514/tcp (open shell), 1099/tcp (open rmiregistry), 1524/tcp (open ingreslock), 2049/tcp (open nfs), 2121/tcp (open cproxy-ftp), 3306/tcp (open mysql), 3632/tcp (open distccd), 5432/tcp (open postgresql), 5900/tcp (open vnc), 6000/tcp (open X11), and 6667/tcp (open irc).

```
kali@kali: ~  
$ sudo nmap -p- 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) 26-09-02 07:04 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.0032s latency).  
Not shown: 65505 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  cproxy-ftp  
3306/tcp  open  mysql  
3632/tcp  open  distccd  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11  
6667/tcp  open  irc
```

The command was run successfully, as shown in Figure 10. Scanning all 65,535 ports found 6 additional open ports beyond the default scan: 3632 (distccd), 6697 (ircs-u), 8787 (msgsrvr), and three high, unnamed ports — 40957, 55566, 56347 — which match the mountd/status services rpcinfo revealed earlier. This shows the default scan alone would have missed real attack surface, especially the distccd service, a tool known for serious remote-code-execution weaknesses.

Step 11: Full TCP + versions

Command used

`sudo nmap -p- -sV <target-ip>`

```

kali-linux-2024.3-vmware-amd64 - VMware Workstation 17 Player
Player 1 2 3 4
kali@kali: ~
File Actions Edit View Help

(kali@kali) ~
$ sudo nmap -p- -sV 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 07:09 EDT
Nmap scan report for 192.168.80.128
Host is up (0.0017s latency).
Not shown: 65505 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
53/tcp    open  domain       ISC BIND 9.4.2
80/tcp    open  http         Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind      2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec         netkit-rsh rexecd
513/tcp   open  login        OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi     GNU Classpath grmiregistry
1524/tcp  open  bindshell    Metasploitable root shell
2049/tcp  open  nfs          2-4 (RPC #100003)
2121/tcp  open  ftp          ProFTPD 1.3.1
3306/tcp  open  mysql        MySQL 5.0.51a-3ubuntu5
3632/tcp  open  distccd     distccd v1 ((GNU) 4.2.4 (Ubuntu 4.2.4-1ubuntu4))
5432/tcp  open  postgresql   PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc          VNC (protocol 3.3)
6000/tcp  open  X11          (access denied)

Type here to search
Windows taskbar: 27°C Light rain, 12:12 PM, 02/09/2026

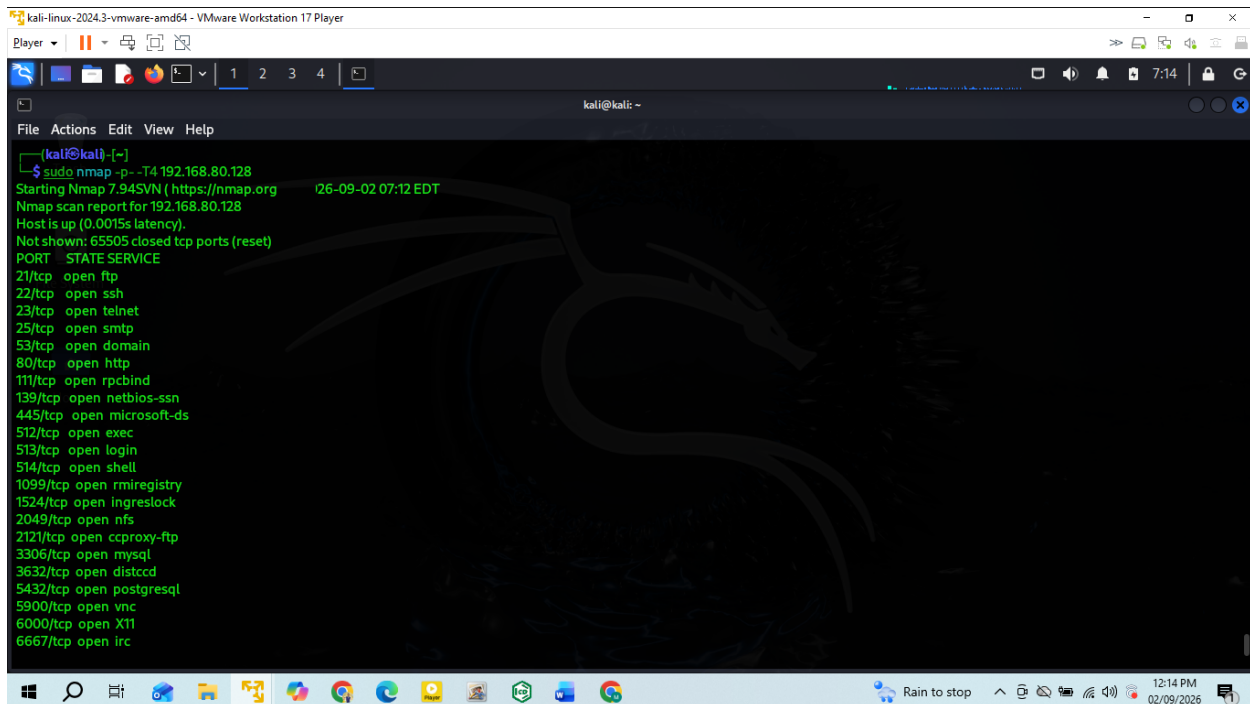
```

The command was run successfully, as shown in Figure 11, completing in 133.56 seconds. This is now the master service inventory: 30 open ports total, with the 6 extra ports from Step 10 now identified — notably distccd v1 (3632), a second UnrealIRCd instance on 6697, and Ruby DRb RMI (8787), plus RPC services (mountd, status, nlockmgr) on the high ports. This confirms a full-port scan combined with version detection gives the most complete picture of the target's real attack surface.

Step 12: Timing template

Command used

`sudo nmap -p- -T4 <target-ip>`



The screenshot shows a terminal window titled 'kali@kali: ~' with a dark background and a dragon logo. The terminal output is as follows:

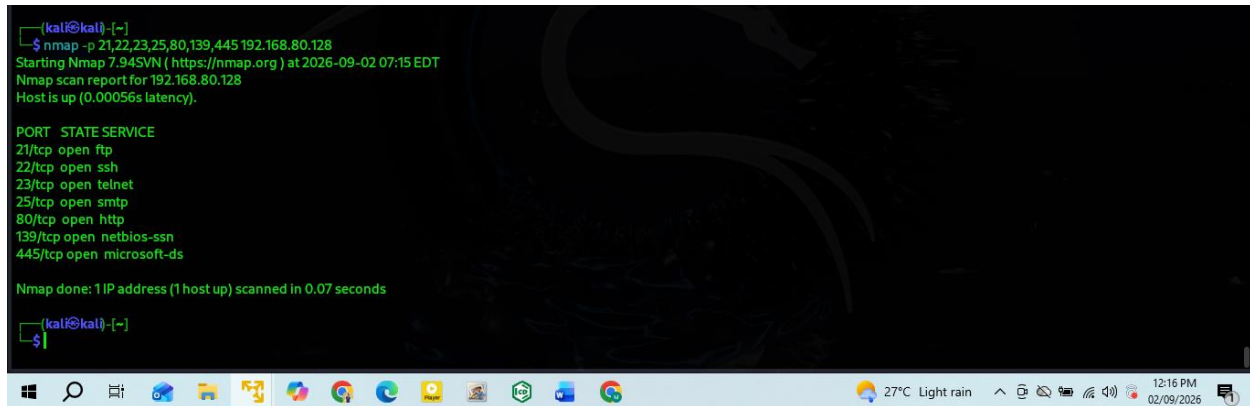
```
kali@kali: ~  
$ sudo nmap -p- -T4 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) 26-09-02 07:12 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.0015s latency).  
Not shown: 65505 closed tcp ports (reset)  
PORT      STATE SERVICE  
21/tcp    open  ftp  
22/tcp    open  ssh  
23/tcp    open  telnet  
25/tcp    open  smtp  
53/tcp    open  domain  
80/tcp    open  http  
111/tcp   open  rpcbind  
139/tcp   open  netbios-ssn  
445/tcp   open  microsoft-ds  
512/tcp   open  exec  
513/tcp   open  login  
514/tcp   open  shell  
1099/tcp  open  rmiregistry  
1524/tcp  open  ingreslock  
2049/tcp  open  nfs  
2121/tcp  open  cproxy-ftp  
3306/tcp  open  mysql  
3632/tcp  open  distccd  
5432/tcp  open  postgresql  
5900/tcp  open  vnc  
6000/tcp  open  X11  
6667/tcp  open  irc
```

The command was run successfully, as shown in Figure 12, completing in 8.65 seconds — almost identical to Step 10's 8.41 seconds. This shows -T4's speed benefit is minimal on a fast, local, low-latency lab network, since the default timing was already near-optimal here; the difference would be more noticeable over a slower or larger network.

Step 13: Selected ports

Command used

```
nmap -p 21,22,23,25,80,139,445 <target-ip>
```



```
(kali@kali) ~$ nmap -p 21,22,23,25,80,139,445 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 07:15 EDT
Nmap scan report for 192.168.80.128
Host is up (0.00056s latency).

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
80/tcp    open  http
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds

Nmap done: 1 IP address (1 host up) scanned in 0.07 seconds

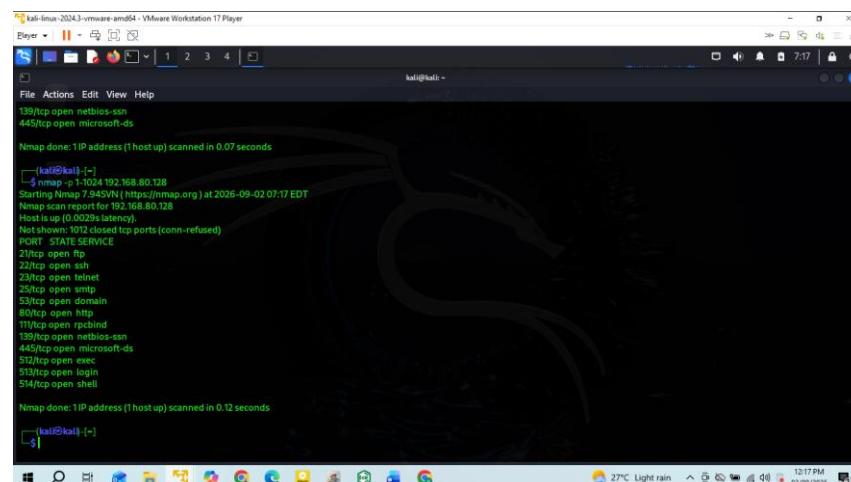
(kali@kali) ~$
```

The command was run successfully, as shown in Figure 13, completing in just 0.07 seconds. All 7 requested ports returned as open, and nothing else was checked. This shows focused scanning is useful when you already know which services you care about — it's much faster than scanning every port and produces a shorter, easier-to-read result.

Step 14: Port range

Command used

```
nmap -p 1-1024 <target-ip>
```



```
(kali@kali) ~$ nmap -p 1-1024 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 07:17 EDT
Nmap scan report for 192.168.80.128
Host is up (0.0028s latency)
Not shown: 1012 closed tcp ports (conn-refused)

PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
23/tcp    open  telnet
25/tcp    open  smtp
53/tcp    open  domain
80/tcp    open  http
111/tcp   open  rpcbind
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds
512/tcp   open  exec
513/tcp   open  login
514/tcp   open  shell

Nmap done: 1 IP address (1 host up) scanned in 0.12 seconds

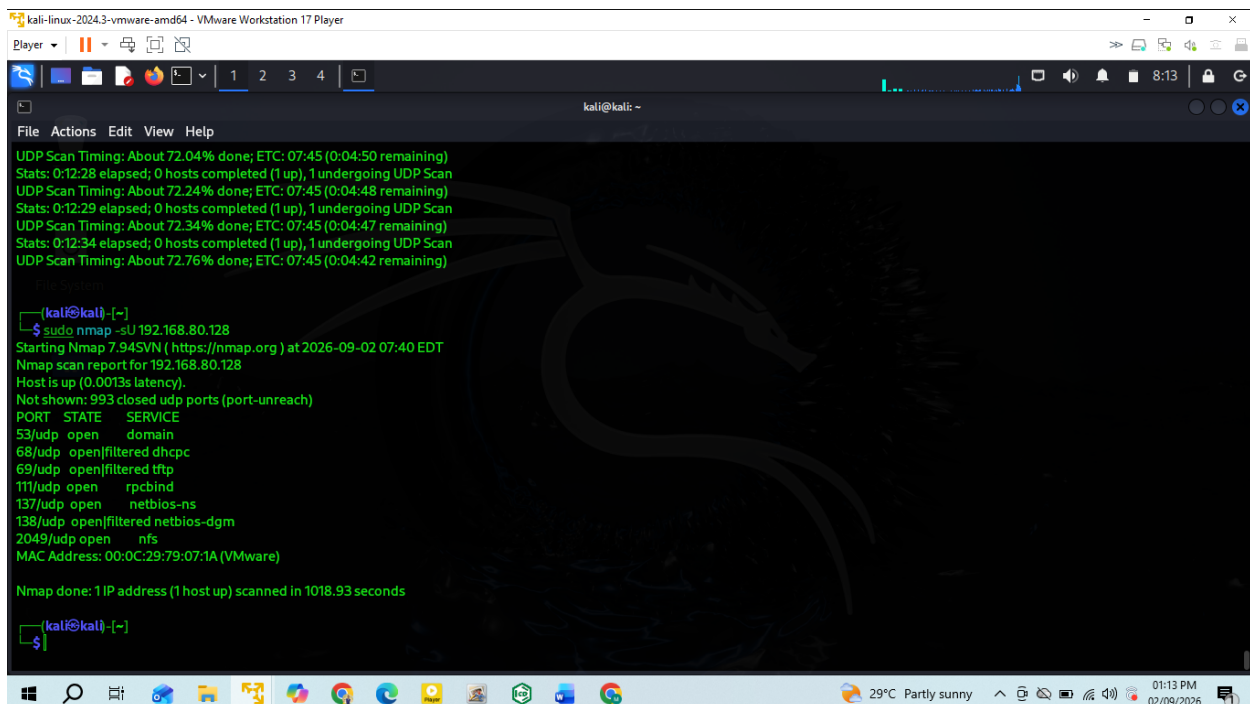
(kali@kali) ~$
```

12 open ports within the 1–1024 range, five more than Step 13's manual list (53, 111, 512, 513, 514). This shows a range scan is useful for catching known-service ports you didn't think to list individually, while still being far faster than a full 65535-port scan.

Step 15: UDP scan

Command used

```
sudo nmap -sU <target-ip>
```



```
kali@kali: ~  
File Actions Edit View Help  
UDP Scan Timing: About 72.04% done; ETC: 07:45 (0:04:50 remaining)  
Stats: 0:12:28 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan  
UDP Scan Timing: About 72.24% done; ETC: 07:45 (0:04:48 remaining)  
Stats: 0:12:29 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan  
UDP Scan Timing: About 72.34% done; ETC: 07:45 (0:04:47 remaining)  
Stats: 0:12:34 elapsed; 0 hosts completed (1 up), 1 undergoing UDP Scan  
UDP Scan Timing: About 72.76% done; ETC: 07:45 (0:04:42 remaining)  
  
[kali@kali] ~  
$ sudo nmap -sU 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 07:40 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.0013s latency).  
Not shown: 993 closed udp ports (port-unreach)  
PORT      STATE      SERVICE  
53/udp    open      domain  
68/udp    open|filtered dhcpd  
69/udp    open|filtered tftp  
111/udp   open      rpcbind  
137/udp   open      netbios-ns  
138/udp   open|filtered netbios-dgm  
2049/udp  open      nfs  
MAC Address: 00:0C:29:79:07:1A (VMware)  
  
Nmap done: 1 IP address (1 host up) scanned in 1018.93 seconds  
  
[kali@kali] ~  
$
```

The command was run successfully, as shown in Figure 15, taking 1018.93 seconds (about 17 minutes) far longer than any TCP scan. It found 7 UDP ports: domain, rpcbind, netbios-ns, and nfs confirmed open, plus dhcpd, tftp, and netbios-dgm as open|filtered, meaning Nmap couldn't be certain whether they're open or blocked. This shows UDP scanning is much slower than TCP and often gives less certain results, since UDP doesn't reply the same way to closed ports.

Step 16: Top UDP ports + versions

Command used

```
sudo nmap -sU --top-ports 20 -sV <target-ip>
```

```

kali@kali: ~
$ sudo nmap -sU --top-ports 20 -sV 192.168.80.128
[sudo] password for kali:
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 08:16 EDT
Nmap scan report for 192.168.80.128
Host is up (0.00081s latency).

PORT      STATE SERVICE VERSION
53/udp    open  domain  ISC BIND 9.4.2
67/udp    closed dhcpd
68/udp    open|filtered dhcpc
69/udp    open|filtered tftp
123/udp   closed ntp
135/udp   closed msrpc
137/udp   open  netbios-ns Microsoft Windows netbios-ns (workgroup: WORKGROUP)
138/udp   open|filtered netbios-dgm
139/udp   closed netbios-ssn
161/udp   closed snmp
162/udp   closed snmptrap
445/udp   closed microsoft-ds
500/udp   closed isakmp
514/udp   closed syslog
520/udp   closed route
631/udp   closed ipp
1434/udp  closed ms-sql-m
1900/udp  closed upnp
4500/udp  closed nat-t-ike
49152/udp closed unknown
MAC Address: 00:0C:29:79:07:1A (VMware)
  
```

The command was run successfully, as shown in Figure 16, completing in 120.45 seconds — much faster than Step 15's full UDP scan. Of the top 20 UDP ports, only 3 (53, 137, 138) were open or open|filtered, with most others cleanly reported as closed. One interesting detail: -sV guessed the OS as Windows here, based on the netbios-ns banner — a false lead worth noting, since we already confirmed via TCP/OS detection that the target is Linux. This shows version-detection guesses can sometimes be misleading and should be cross-checked against other evidence.

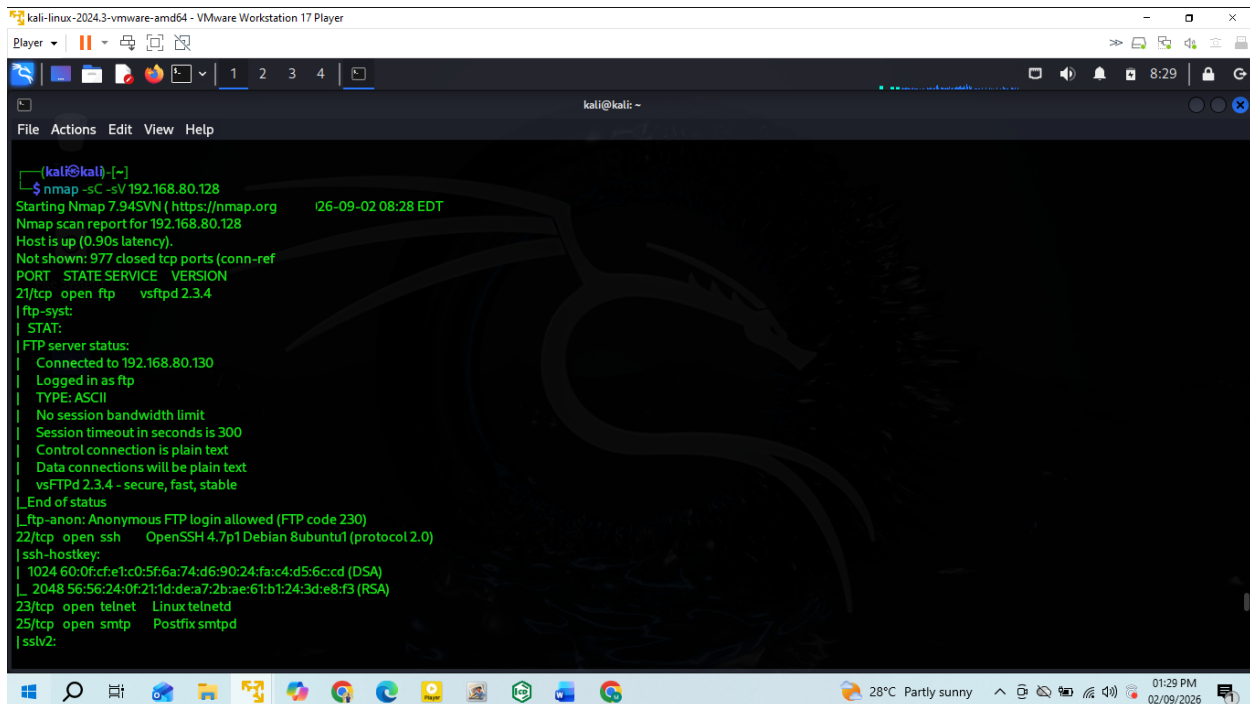
PART 4 - SAFE NMAP NSE ENUMERATION

This part uses safe Nmap Scripting Engine (NSE) scripts to enumerate further detail about the discovered services.

Step 17: Default scripts

Command used

`nmap -sC -sV <target-ip>`



```
kali@kali: ~  
$ nmap -sC -sV 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) 26-09-02 08:28 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.90s latency).  
Not shown: 977 closed tcp ports (conn-ref)  
PORT      STATE SERVICE VERSION  
21/tcp    open  ftp      vsftpd 2.3.4  
|_ftp-syst:  
|_STAT:  
|_FTP server status:  
|_  Connected to 192.168.80.130  
|_  Logged in as ftp  
|_  TYPE: ASCII  
|_  No session bandwidth limit  
|_  Session timeout in seconds is 300  
|_  Control connection is plain text  
|_  Data connections will be plain text  
|_  vsFTPD 2.3.4 - secure, fast, stable  
|_End of status  
|_ftp-anon: Anonymous FTP login allowed (FTP code 230)  
22/tcp    open  ssh      OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)  
|_ssh-hostkey:  
|_ 1024 60:0f:cf:e1:c0:5f:6a:74:d6:90:24:fa:c4:d5:6c:cd (DSA)  
|_ 2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)  
23/tcp    open  telnet   Linux telnetd  
25/tcp    open  smtp     Postfix smtpd  
|_sslv2:
```

The command was run successfully, as shown in Figure 17, completing in 24.35 seconds. Two useful script findings: ftp-anon confirmed anonymous FTP login is allowed (code 230), and smb-os-discovery revealed the exact OS (Unix, Samba 3.0.20-Debian), computer name, and domain. This shows default scripts add real depth beyond port/version data alone, pulling out configuration details an attacker could use.

Step 18: HTTP title

Command used

`nmap -p 80 --script http-title <target-ip>`

```

kali@kali: ~
File Actions Edit View Help
_ message_signing: disabled (dangerous, but default)
_ clock-skew: mean: 1h00m21s, deviation: 2h00m00s, median: 21s
_ smb2-time: Protocol negotiation failed (SMB2)
| smb-os-discovery:
| OS: Unix (Samba 3.0.20-Debian)
| Computer name: metasploitable
| NetBIOS computer name:
| Domain name: localdomain
| FQDN: metasploitable.localdomain
| System time: 2026-09-02T08:29:09-04:00
|_ nbstat: NetBIOS name: METASPLOITABLE, NetBIOS user: <unknown>, NetBIOS MAC: <unknown> (unknown)

Service detection performed. Please report any incorrect results at https://nmap.org/submit/
Nmap done: 1 IP address (1 host up) scanned in 24.35 seconds

(kali@kali) [~]
$ nmap -p 80 --script http-title 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 08:31 EDT
Nmap scan report for 192.168.80.128
Host is up (0.00055s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-title: Metasploitable2 - Linux

Nmap done: 1 IP address (1 host up) scanned in 0.25 seconds

(kali@kali) [~]
$

```

The command was run successfully, as shown in Figure 18. The page title returned was "Metasploitable2 - Linux" — a direct clue confirming both the platform and OS from the web layer alone, without needing a browser.

Step 19: HTTP headers

Command used

`nmap -p 80 --script http-headers <target-ip>`

```

kali@kali: ~
File Actions Edit View Help
Host is up (0.00055s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-title: Metasploitable2 - Linux

Nmap done: 1 IP address (1 host up) scanned in 0.25 seconds

(kali@kali) [~]
$ nmap -p 80 --script http-headers 192.168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 08:36 EDT
Nmap scan report for 192.168.80.128
Host is up (0.00032s latency).

PORT      STATE SERVICE
80/tcp    open  http
|_ http-headers:
| Date: Wed, 02 Sep 2026 12:36:25 GMT
| Server: Apache/2.2.8 (Ubuntu) DAV/2
| X-Powered-By: PHP/5.2.4-Zubuntu5.10
| Connection: close
| Content-Type: text/html
|_ (Request type: HEAD)

Nmap done: 1 IP address (1 host up) scanned in 0.22 seconds

(kali@kali) [~]
$

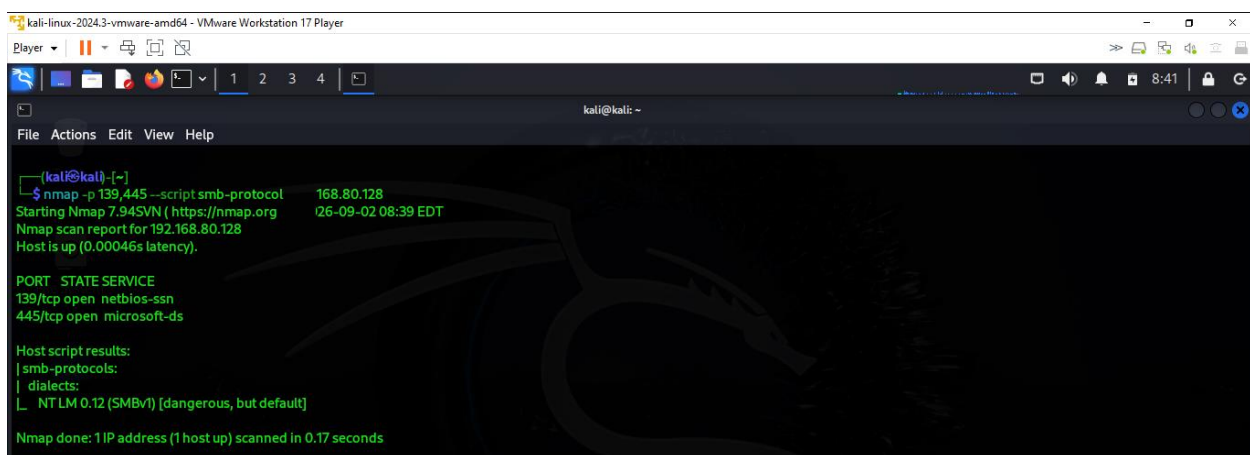
```

The command was run successfully, as shown in Figure 19. The response revealed the Apache/2.2.8 (Ubuntu) server banner and, notably, X-Powered-By: PHP/5.2.4-2ubuntu5.10 — an outdated PHP version disclosed directly in the headers. This shows how HTTP headers can leak exact software versions useful for further assessment.

Step 20: SMB protocols

Command used

```
nmap -p 139,445 --script smb-protocols <target-ip>
```



```
kali-linux-2024.3-vmware-amd64 - VMware Workstation 17 Player
Player
File Actions Edit View Help
kali@kali: ~
(kali@kali)~$ nmap -p 139,445 --script smb-protocols 168.80.128
Starting Nmap 7.94SVN ( https://nmap.org ) 26-09-02 08:39 EDT
Nmap scan report for 192.168.80.128
Host is up (0.00046s latency).

PORT      STATE SERVICE
139/tcp   open  netbios-ssn
445/tcp   open  microsoft-ds

Host script results:
| smb-protocols:
|   dialects:
|_  NT LM 0.12 (SMBv1) [dangerous, but default]

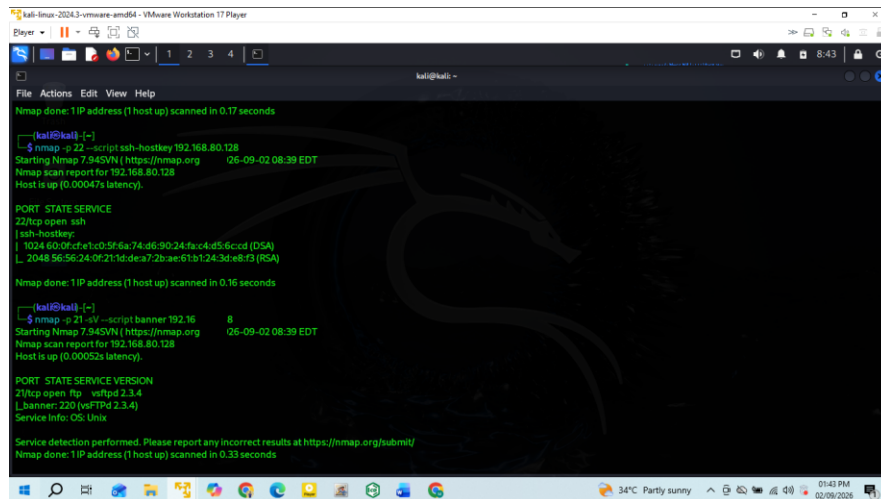
Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds
```

The command was run successfully, as shown in Figure 20. Only one dialect was supported: NT LM 0.12 (SMBv1), flagged by Nmap itself as "dangerous, but default." This shows the target still runs the legacy, insecure SMBv1 protocol.

Step 21: SSH host keys

Command used

```
nmap -p 22 --script ssh-hostkey <target-ip>
```

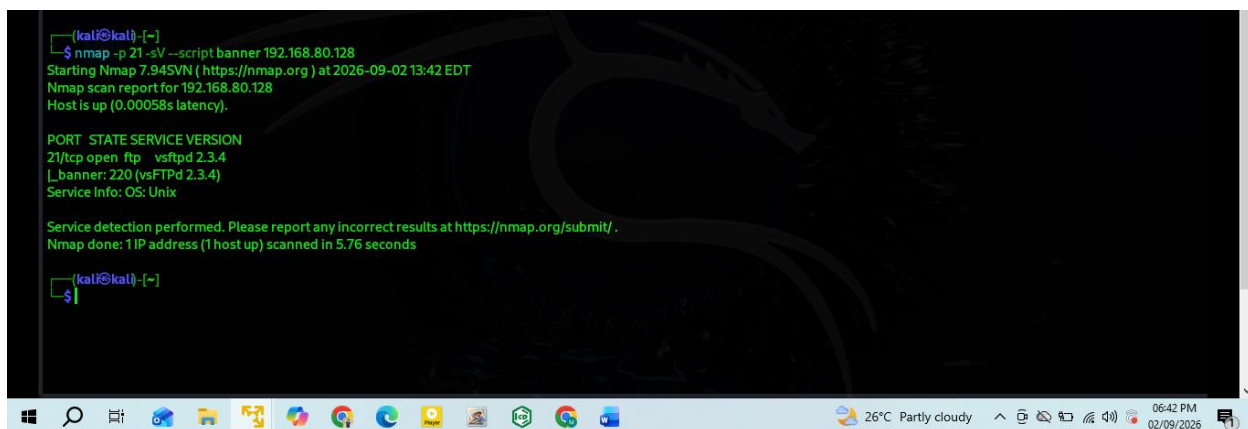


```
kali@kali: ~  
$ nmap -p 22 --script ssh-hostkey 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) at 26-09-02 08:39 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.00047s latency).  
  
PORT STATE SERVICE  
22/tcp open  ssh  
ssh-hostkey:  
1024 60:0f:ef:1c:05:6a:74:d6:90:24:fc:d4:5f:6cc0 (DSA)  
2048 56:56:24:0f:21:1d:de:a7:2b:ae:61:b1:24:3d:e8:f3 (RSA)  
  
Nmap done: 1 IP address (1 host up) scanned in 0.17 seconds  
  
kali@kali: ~  
$ nmap -p 21 -sV --script banner 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) at 26-09-02 08:39 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.00052s latency).  
  
PORT STATE SERVICE VERSION  
21/tcp open  ftp      vsftpd 2.3.4  
_banner: 220 (vsFTPd 2.3.4)  
Service Info: OS: Unix  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/  
Nmap done: 1 IP address (1 host up) scanned in 0.33 seconds
```

Step 22: FTP banner

Command used

`nmap -p 21 -sV --script banner <target-ip>`



```
kali@kali: ~  
$ nmap -p 21 -sV --script banner 192.168.80.128  
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-09-02 13:42 EDT  
Nmap scan report for 192.168.80.128  
Host is up (0.00058s latency).  
  
PORT STATE SERVICE VERSION  
21/tcp open  ftp      vsftpd 2.3.4  
_banner: 220 (vsFTPd 2.3.4)  
Service Info: OS: Unix  
  
Service detection performed. Please report any incorrect results at https://nmap.org/submit/.  
Nmap done: 1 IP address (1 host up) scanned in 5.76 seconds  
  
kali@kali: ~  
$
```

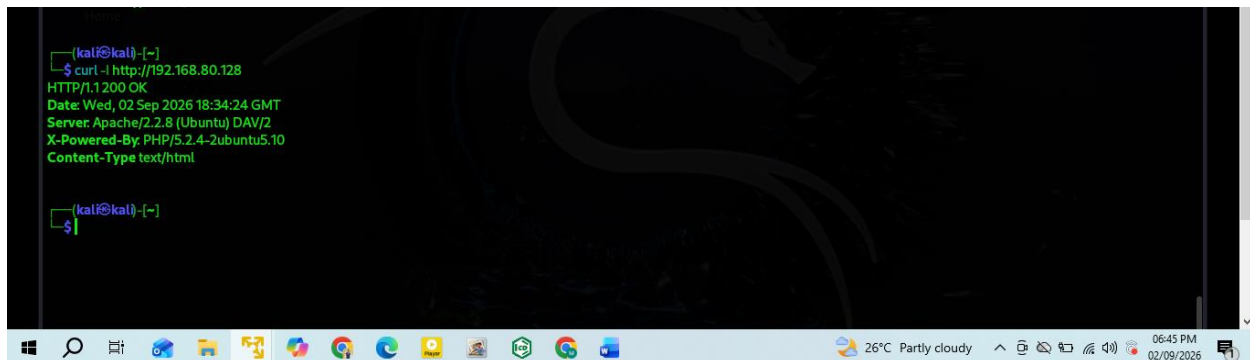
PART 5 - MANUAL HTTP CONFIRMATION

This part manually confirms what the target's web service returns, independent of any automated scanner.

Step 23: Request headers with curl

Command used

```
curl -I http://<target-ip>
```

A screenshot of a Kali Linux terminal window. The prompt is '(kali@kali)~'. The user has entered the command '\$ curl -I http://192.168.80.128'. The output is: 'HTTP/1.1 200 OK', 'Date: Wed, 02 Sep 2026 18:34:24 GMT', 'Server: Apache/2.2.8 (Ubuntu) DAV/2', 'X-Powered-By: PHP/5.2.4-2ubuntu5.10', and 'Content-Type: text/html'. The terminal has a dark background with a faint Kali Linux logo. The Windows taskbar is visible at the bottom with various icons and a system tray showing '26°C Partly cloudy' and the time '06:45 PM 02/09/2026'.

The command was run successfully, as shown in Figure 23. It returned status **200 OK** with Server: Apache/2.2.8 (Ubuntu) DAV/2, matching Nmap's findings exactly. This confirms the web service details without needing a scanner — a simple manual cross-check.

PART 6 - WHATWEB FINGERPRINTING

WhatWeb was used purely as a technology-fingerprinting tool in this lab; deeper web-application testing tools are reserved for Lab 2.

Step 24: Basic fingerprint

Command used

whatweb http://<target-ip>

```

(kali@kali) ~-
$ whatweb http://192.168.80.128

http://192.168.80.128 OK[Apache/2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, IP[192.168.80.128], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux, WebDAV/2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

(kali@kali) ~-
$

```

The command was run successfully, as shown in Figure 24. WhatWeb identified Apache 2.2.8, Ubuntu Linux, PHP 5.2.4-2ubuntu5.10, WebDAV, and the page title, all in one compact line. This matches everything found separately by Nmap and curl, confirming the technology stack from a single dedicated tool.

Step 25: Verbose output

Command used

whatweb -v http://192.168.80.128

```

(kali@kali) ~-
$ whatweb http://192.168.80.128

http://192.168.80.128 OK[Apache/2.2.8], Country[RESERVED][ZZ], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, IP[192.168.80.128], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux, WebDAV/2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

(kali@kali) ~-
$ whatweb -v http://192.168.80.128

WhatWeb report for http://192.168.80.128
Status : 200 OK
Title : Metasploitable2 - Linux
IP : 192.168.80.128
Country : RESERVED, ZZ

Summary : Apache/2.2.8, HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, PHP[5.2.4-2ubuntu5.10, WebDAV/2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

Detected Plugins:
[ Apache ]
The Apache HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows NT. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards.
Version : 2.2.8 (from HTTP Server Header)
Google Dorks (8)
Website : http://httpd.apache.org/

[ HTTPServer ]
HTTP server header string. This plugin also attempts to identify the operating system from the server header.

```

The command was run successfully, as shown in Figure 25. Compared to Step 24's compact one-line summary, verbose mode added plugin descriptions, Google Dork counts, and the full raw

HTTP headers including Content-Length. This shows verbose mode is useful for understanding *why* each technology was detected, not just what was detected.

Step 26: Aggression level 1

Command used

```
(kali@kali)~$ whatweb -a 1 http://192.168.80.128
http://192.168.80.128[200 OK] Apache/2.2.8 (Ubuntu) DAV/2, IP[192.168.80.128], PHP[5.2.4-2ubuntu5.10], Title[Meta], itable2 - Linux, WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
(kali@kali)~$
```

The command was run successfully, as shown in Figure 26. Level 1 returned the same compact summary as the default scan in Step 24, confirming level 1 is the lightest, minimal-request mode.

Step 27: Aggression level 3

Command used

`whatweb -a 3 -v http://<target-ip>`

```
(kali@kali)~$ whatweb -a 3 -v http://192.168.80.128
WhatWeb report for http://192.168.80.128
Status : 200 OK
Title : Metasploit2 - Linux
IP : 192.168.80.128
Country : RESERVED,ZZ

Summary : Apache/2.2.8, HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, PHP[5.2.4-2ubuntu5.10, WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

Detected Plugins:
[ Apache ]
The Apache HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows NT. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards.
Version : 2.2.8 (from HTTP Server Header)
Google Dorks: (3)
Website : http://httpd.apache.org/

[ HTTPServer ]
HTTP server header string. This plugin also attempts to identify the operating system from the server header.
OS : Ubuntu Linux
String : Apache/2.2.8 (Ubuntu) DAV/2(from server string)
```

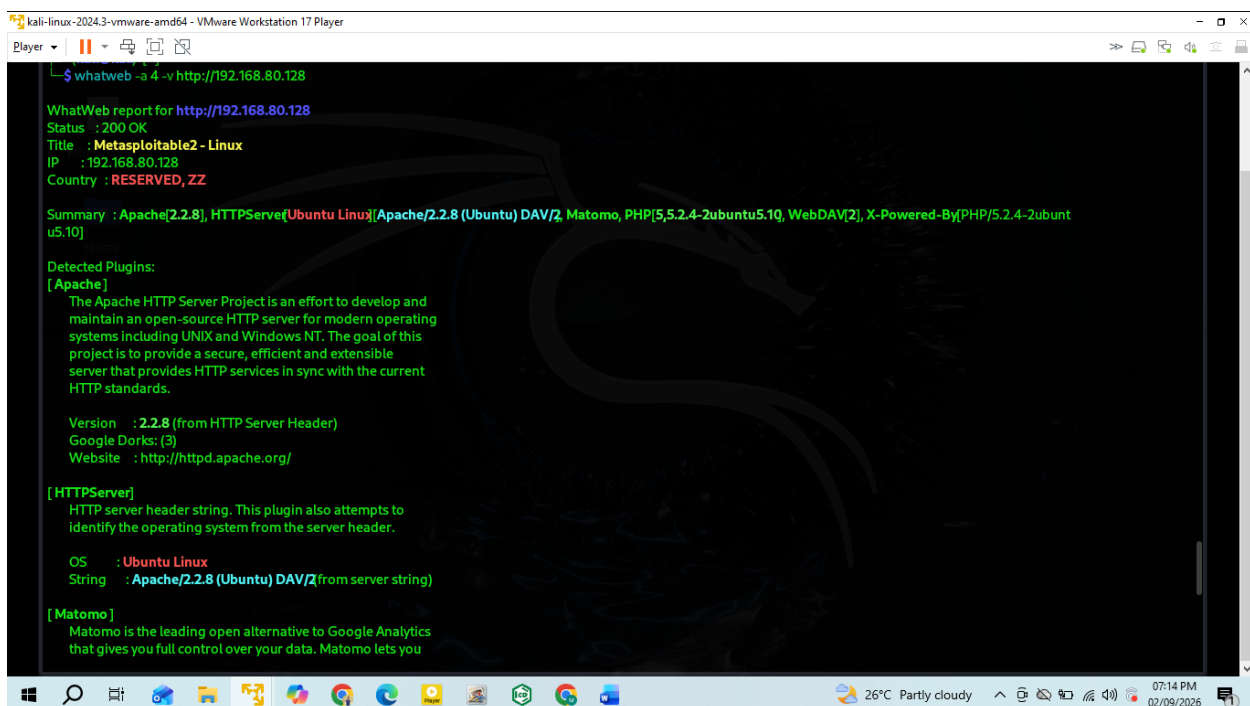
The command was run successfully, as shown in Figure 27. Compared to level 1, level 3 made

extra requests and found one additional detail: a second PHP version indicator (Version: 5) alongside the full version string, from a different detection method. This shows higher aggression levels can confirm findings through multiple techniques, though the core technology stack stayed the same.

Step 28: Aggression level 4

Command used

```
whatweb -a 4 -v http://<target-ip>
```



```
$ whatweb -a 4 -v http://192.168.80.128

WhatWeb report for http://192.168.80.128
Status : 200 OK
Title : Metasploitable2 - Linux
IP : 192.168.80.128
Country : RESERVED, ZZ

Summary : Apache[2.2.8], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, Matomo, PHP[5.5.2.4-2ubuntu5.10, WebDAV/2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

Detected Plugins:
[ Apache ]
  The Apache HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows NT. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards.

  Version : 2.2.8 (from HTTP Server Header)
  Google Dorks: (3)
  Website : http://httpd.apache.org/

[ HTTPServer ]
  HTTP server header string. This plugin also attempts to identify the operating system from the server header.

  OS : Ubuntu Linux
  String : Apache/2.2.8 (Ubuntu) DAV/2 (from server string)

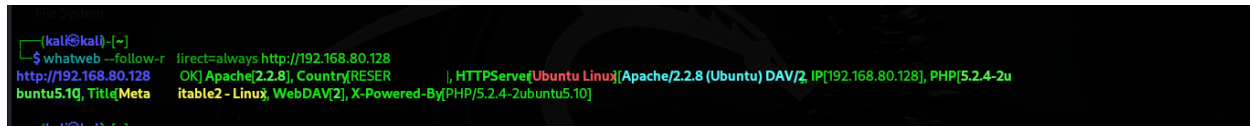
[ Matomo ]
  Matomo is the leading open alternative to Google Analytics that gives you full control over your data. Matomo lets you
```

The command was run successfully, as shown in Figure 28. Level 4 found one new item beyond levels 1 and 3: a **Matomo** (analytics) plugin signature, made possible by its heavier, more aggressive probing. This confirms level 4 does add new information for this target, at the cost of more requests sent.

Step 29: Follow redirects

Command used

`whatweb --follow-redirect=always http://<target-ip>`



```
(kali@kali) ~$ whatweb --follow-redirect=always http://192.168.80.128
http://192.168.80.128 OK] Apache[2.2.8], Country[RESERVED], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, IP[192.168.80.128], PHP[5.2.4-2ubuntu5.10], Title[Metasploitable2 - Linux], WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]
```

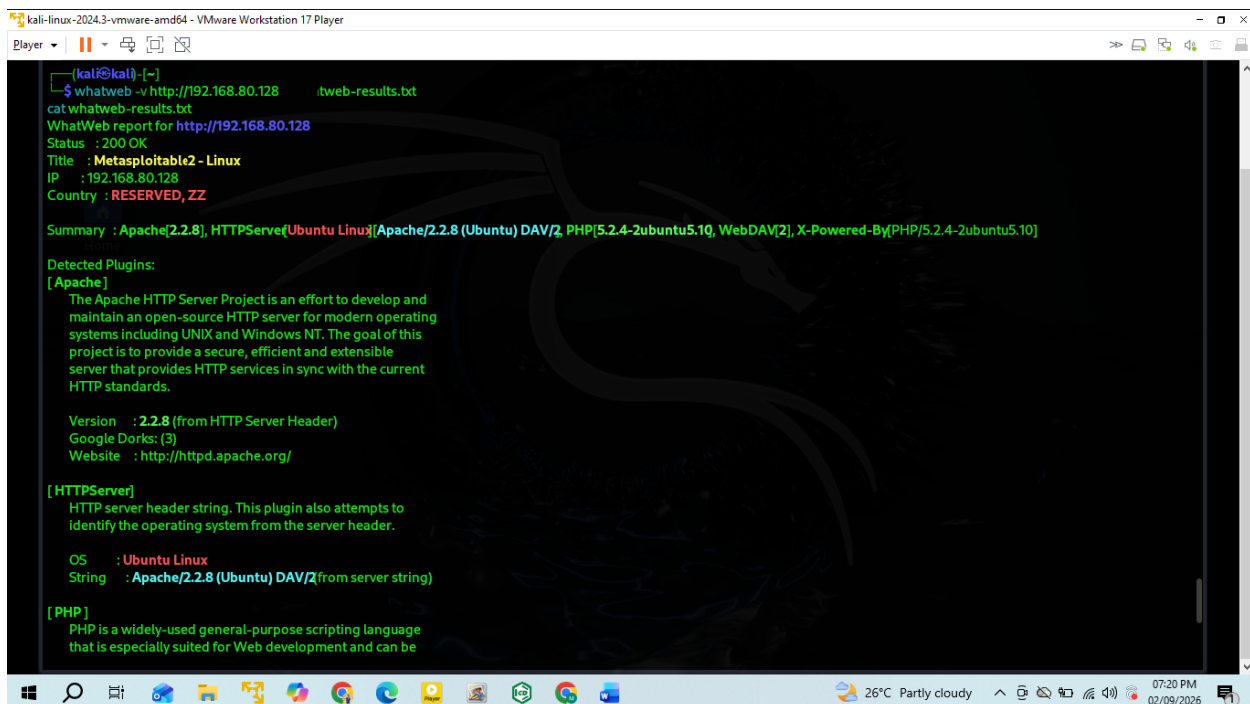
The command was run successfully, as shown in Figure 29. The result was identical to the default scan, with no change in status or URL — showing no redirect occurred on this page.

Step 30: Save results

Command used

`whatweb -v http://<target-ip> > whatweb-results.txt`

`cat whatweb-results.txt`



```
(kali@kali) ~$ whatweb -v http://192.168.80.128 > whatweb-results.txt
cat whatweb-results.txt
WhatWeb report for http://192.168.80.128
Status : 200 OK
Title : Metasploitable2 - Linux
IP : 192.168.80.128
Country : RESERVED, ZZ

Summary : Apache[2.2.8], HTTPServer[Ubuntu Linux][Apache/2.2.8 (Ubuntu) DAV/2, PHP[5.2.4-2ubuntu5.10, WebDAV[2], X-Powered-By[PHP/5.2.4-2ubuntu5.10]

Detected Plugins:
[ Apache ]
The Apache HTTP Server Project is an effort to develop and maintain an open-source HTTP server for modern operating systems including UNIX and Windows NT. The goal of this project is to provide a secure, efficient and extensible server that provides HTTP services in sync with the current HTTP standards.

Version : 2.2.8 (from HTTP Server Header)
Google Dorks: (3)
Website : http://httpd.apache.org/

[ HTTPServer ]
HTTP server header string. This plugin also attempts to identify the operating system from the server header.

OS : Ubuntu Linux
String : Apache/2.2.8 (Ubuntu) DAV/2 (from server string)

[ PHP ]
PHP is a widely-used general-purpose scripting language that is especially suited for Web development and can be
```

The command was executed successfully as shown in Figure 30 above. `>` redirects output into a file; `cat` then displays that file's content. A saved text report matching the terminal output.

Service Inventory Table

Port	Protocol	State	Service	Version/Evidence	Command
21	TCP	Open	ftp	vsftpd 2.3.4; anon login allowed	-sV, -sC
22	TCP	Open	ssh	OpenSSH 4.7p1 Debian 8ubuntu1	-sV
23	TCP	Open	telnet	Linux telnetd	-sV
25	TCP	Open	smtp	Postfix smtpd	-sV
53	TCP/UDP	Open	domain	ISC BIND 9.4.2	-sV, -sU
80	TCP	Open	http	Apache 2.2.8 (Ubuntu) DAV/2, PHP 5.2.4	-sV, whatweb
111	TCP	Open	rpcbind	RPC #100000	-sC
139/445	TCP	Open	netbios-ssn/microsoft-ds	Samba 3.0.20-Debian, SMBv1 only	-sV, smb-protocols
512-514	TCP	Open	exec/login/shell	rexecd, rlogind, tcpwrapped	-sV
1099	TCP	Open	java-rmi	GNU Classpath grmiregistry	-sV
1524	TCP	Open	bindshell	"Metasploitable root shell"	-sV
2049	TCP/UDP	Open	nfs	RPC #100003	-sV, -sU
2121	TCP	Open	ftp	ProFTPD 1.3.1	-sV
3306	TCP	Open	mysql	MySQL 5.0.51a-3ubuntu5	-sV
3632	TCP	Open	distccd	v1, GNU 4.2.4	-p-
5432	TCP	Open	postgresql	PostgreSQL 8.3.0-8.3.7	-sV
5900	TCP	Open	vnc	protocol 3.3	-sV
6000	TCP	Open	X11	access denied	-sV
6667/6697	TCP	Open	irc	UnrealIRCd 3.2.8.1	-sV
8009	TCP	Open	ajp13	Apache JServ 1.3	-sV
8180	TCP	Open	http	Apache Tomcat/Coyote 1.1	-sV
8787	TCP	Open	drb	Ruby DRb RMI	-p-
137/138	UDP	Open	netbios-ns/dgm	Windows-named (misleading)	-s

DISCUSSION

Looking at everything gathered across the seven parts of this lab, Metasploitable 2 turns out to have a surprisingly wide attack surface for a single machine — 23 services showed up under a default scan, and that number climbed to 30 once every TCP port was checked, with another 7 UDP services on top. A good chunk of what's running is old and insecure by today's standards: telnet, the rsh/rlogin trio on ports 512-514, and FTP with no encryption. A few port labels even seem to advertise trouble directly, like port 1524 identifying itself as a "Metasploitable root shell." That said, none of this was tested further — spotting an open port or a suspicious label is not the same as proving something is actually breakable, so everything here stays at the level of an observation rather than a confirmed vulnerability.

ANALYSIS AND FINDINGS

Every scan type used default, -sV, -O, -A, the full port sweep, and UDP pointed to the same conclusion: this is a Linux 2.6.x Metasploitable box running 30 TCP and 7 UDP services. A handful of results stood out above the rest: FTP allowed anonymous logins on vsftpd 2.3.4, SMB was left with signing turned off, and the jump from 23 to 30 open ports once -p- ran uncovered services the quick scan never touched distccd, a second copy of UnrealIRCd, and a Ruby DRb interface. What's reassuring is how often the different tools agreed with each other. The FTP banner grab lined up exactly with the -sV version string, and curl's headers matched what WhatWeb reported independently. The one place things diverged was the UDP -sV pass, which read the NetBIOS response and guessed "Windows" clearly wrong, since TCP-based OS detection had already pinned the box as Linux. That mismatch is a good example of why a single tool's output shouldn't be taken at face value without checking it against other evidence.

Pulling it together, the picture is of a system that's been deliberately left outdated and loosely configured across many services at once exactly what you'd expect from a machine built for training rather than one meant to be run in production.

CHALLENGES AND SOLUTIONS

A handful of small hiccups came up while working through the steps. The UDP scan alone ran for roughly 17 minutes, which felt slow compared to every TCP scan finishing in seconds — but that's simply how UDP scanning behaves, not something to troubleshoot, so the only real "fix" was letting it run to completion. Separately, a stray ~ got typed onto the end of the target IP in one command, which made Nmap unable to resolve the host at all; retyping the line correctly solved it right away. WhatWeb also got interrupted partway through with Ctrl+C by accident and had to be started over. None of these changed the accuracy of the final results, but they were a useful reminder to read a command over before hitting enter, and to expect some scan types to simply take longer than others.

CONCLUSION

This lab demonstrated the full reconnaissance workflow — from host discovery through service/version detection, OS fingerprinting, full-port and UDP scanning, NSE enumeration, and web technology fingerprinting — and showed how each tool and technique adds a different layer of detail to the overall picture of a target. It also reinforced the discipline of treating findings as evidence-based observations rather than assumptions. This groundwork directly feeds into Lab 2, where the web service identified here (Apache/PHP on port 80) becomes the actual target for hands-on web application security testing.

RECOMMENDATIONS

1. Take a VM snapshot before testing begins, so any misstep during later, more invasive labs can be rolled back cleanly.
2. Keep evidence organised as you go — save raw output to text files (as done with whatweb-results.txt) rather than relying only on screenshots, since raw text is easier to reference and quote accurately when writing the report later.

STUDENT QUESTIONS

1. What is reconnaissance?

Reconnaissance is the information-gathering phase carried out before any deeper testing. It involves identifying live hosts, open ports, running services and their versions, without exploiting anything.

2. What is the difference between host discovery and port scanning?

Host discovery (nmap -sn) identifies which IP addresses are alive on a network. Port scanning goes further, checking a specific live host to see which ports are open.

3. Explain open, closed and filtered.

Open means a service is actively accepting connections on that port. Closed means the host is reachable but nothing is listening. Filtered means a firewall or filter prevented Nmap from determining the true state.

4. What does -sV do?

-sV sends version-detection probes to open ports and attempts to identify the exact product and version of the software providing the service.

5. What does -O do?

-O enables Nmap's operating-system fingerprinting, which analyses TCP/IP stack behaviour to estimate the target's operating system and give a confidence level.

6. Explain what -A combines and why it is noisier.

-A combines OS detection, service/version detection, default NSE scripts and traceroute in a single scan. It is noisier because it sends significantly more probes and requests than a basic scan.

7. What does -p- mean?

-p- instructs Nmap to scan all 65535 TCP ports instead of only the default set of common ports.

8. Why can UDP scans take longer?

UDP has no handshake, so when no response is received Nmap often has to wait for a timeout before it can report a port as open|filtered, which slows the scan considerably.

9. What does -sC do?

-sC runs Nmap's default set of safe NSE (Nmap Scripting Engine) scripts against the detected services to gather additional information.

10. What information did the HTTP title/headers scripts reveal?

The http-title script (Step 18) revealed the page title: "Metasploitable2 - Linux." The http-headers script (Step 19) revealed the Server header Apache/2.2.8 (Ubuntu) DAV/2, the X-Powered-By: PHP/5.2.4-2ubuntu5.10 header, a Content-Type: text/html header, and the response date. Together these confirmed the exact web server software, OS, and scripting language version, all without opening a browser.

11. What did SMB/SSH/FTP enumeration add?

The smb-protocols script (Step 20) showed the target only supports the SMBv1 dialect (NT LM 0.12), which Nmap itself flags as "dangerous, but default." The ssh-hostkey script (Step 21) revealed two host key algorithms: a 1024-bit DSA key and a 2048-bit RSA key. The FTP banner check (Step 22) returned 220 (vsFTPD 2.3.4), which matched the version already reported by -sV, confirming that result was accurate. Together, these added protocol-level and identity details that go beyond a basic port/version scan.

12. What is the difference between Nmap service detection and WhatWeb fingerprinting?

Nmap service detection identifies services and versions at the network/protocol level (e.g. what is listening on port 80). While WhatWeb fingerprints the web application layer itself - the CMS, JavaScript libraries, and server software revealed by the HTTP response content.

13. How did WhatWeb levels 1, 3 and 4 differ?

Level 1 (Step 26) returned the same compact result as the default scan just the core technologies (Apache, PHP, WebDAV, page title). Level 3 (Step 27) made extra requests and confirmed the PHP version through a second, separate detection method, though no new technology appeared. Level 4 (Step 28) was the only one to reveal something new: a Matomo analytics plugin, detected only because of its heavier, more aggressive probing. None of the levels took noticeably longer to run in this lab environment — the difference was in depth of detection, not scan time.

14. Why must aggressive scanning/fingerprinting remain inside the authorised lab?

Aggressive scans generate significant network traffic and could be mistaken for, or genuinely cause, disruption on a real network. Restricting them to an authorised, isolated lab avoids legal and ethical problems and prevents unintended damage.

Part 10 - Completion Checklist

- ✓☐ I used only the authorised target.
- ✓☐ I completed all required Nmap scan types.
- ✓☐ I can explain each important option.
- ✓☐ I completed safe NSE enumeration.
- ✓☐ I compared WhatWeb aggression levels.
- ✓☐ I built a final service inventory.
- ✓☐ I answered all questions in my own words.

REFERENCES

International Cybersecurity and Digital Forensics Academy. (2026). WADF104 - Cybersecurity Foundations (WADF-2026-M01) Student Laboratory Workbook.

Kali Linux Documentation Team. (2026). Kali Linux Documentation. Retrieved from <https://www.kali.org/docs/>

Nmap Project. (2026). Nmap Reference Guide. Retrieved from <https://nmap.org/book/man.html>